Things to Know for this Lab

All writing and reading requires including stdio.h:

```
#include<stdio.h>   /* this is a comment line (can be on separate line too) */
```

Basic structure of a C program (call it foo.c):

```
#include….
int main(){     /* start of the main function. All execution starts here */
        ….      /* code */
        return (...)      /* return value to the operating system */
}               /* end of the main program */
```

To compile the code: gcc foo.c -o foo compiles the program **foo.c** and creates the executable **foo**.

Most programs we write will also require math.h, which is the include file for the libmath.so library. The 'so' indicates that the library is a "shared" library, which means that a single copy of the library function is in the memory which is used by all programs that require it.
To compile such a program, we have to teach the compiler that we need libmath.so For that we execute: **gcc foo.c -lm -o foo**. (Actually the current gcc on my computer seems to use libmath.so automatically. So the -lm may not be required.)

Printf:

To write something, use printf("...").

```
printf("This is a line\n");
This is a line
```

To print multi-line output

```
printf("This is\na line\n");  /* each \n starts a new line */
This is
a line
```

Note: if you had written

```
printf("This is\n a line\n");
```

A space would have been added ahead of the 'a' in the second line.

Integer variables and assignment and printing the same:

To declare an integer variable and assign it a value

```
int a;
a=4;
```

Can also assign the value in the same line:

```
int a=4;
```

To write something that includes an integer variable (printf conversion specification for integer is %d)

```
printf("The value of a = %d\n",a); /* output printed, with %d replaced by value */
The value of a = 4
```

Note that the following is allowed: %6d - print as integer, using 6 spaces

```
int a=8, b=120;
printf("The value of a = %3d and b = %3d\n",8,15); /* use 2 digits for all output */
The value of a =  8 and b = 12                      /* note extra space for 8 */
```

Addition of variables and constants

```
Int b=10;
Int c=15;   /* can give the variable a value inside the declaration itself */
Int d,e;
d=a+b+c;  /* addition of variables */
e=a+4+c;  /* addition of variables and constants (here 4 is a constant) */
```

String constants, declaration and output.

To declare a string variable, and assign it a value

```
char name[ ] = "Abc Xyz";
```

The syntax will be explained later, but this is a simple way to declare string constants. To print some strings, use

```
printf("My name is %s, and my home is in %s\n", name, city);
```

%s is the symbol used to indicate a string. It will be taken from the list of variables following the format string. Again, we can modify the output of the string:

```
printf("My name is %10s, and my home is in %10s\n", name, city);
```

This limits the number of spaces to be used for name and city to be a maximum of 10. If the string has 10 spaces or less, it will print out correctly with additional spaces as required. But if the string is larger, C will print out the whole string so that the output is readable.

Float and Double variables, assignment and output:

To declare a real number, and assign it a value

```
float a = 4.5;
```

This creates a real number that uses 32 bits of storage. The mantissa is 24 bits in size including sign, and the exponent is 8 bits in size. Note that exponent cannot exceed 7 bits as there is an offset. To print float numbers use

```
printf("I am %f cm tall\n", 66.5);
```

Note that the value for height can also be taken from a constant

Long long int variables:

To declare a long int variable. This is a little strange. "int" is not a fixed size. It depends on the cpu of the computer. So a long integer variable is called "long long int"

```
long long int aa = 123456789;
printf("aa = %lld\n", aa);
```

This creates an integer that uses 64 bits of storage. Such an integer can store much larger size integer values. For example,

```
long long int amnt=1000;    /* amount in crores to be printed out */
Int crore = 10000000;       /* Number of Rs in 1 crore */
amnt = amnt*crore;          /* amnt converted to Rs. */
printf("1000 crores is %lld Rs.", amt );
```

To declare a long real number, and assign it a value

```
double b = 3.5e250
```

This creates large real numbers whose mantissa or exponent will not fit into a float variable.

Arrays

Arrays are a sequence of numbers or strings. For example:

```
int a=[1,2,3,4,5];      /* array allocated and initialized */
int b[10];              /* array allocated but values not given */
```

The type of variable is not constrained. It can be anything. For example we can have strings, which are arrays of characters. We can also have arrays of strings, which means it is an array of arrays of char. Etc.

Arrays are very frequently used together with for loops.

Scanf:

If we want the user to enter some information, we use the scanf function. The format is

```
scanf("format",&arg1, &arg2, …, &argn);
```

For example, (assuming #include<stdio.h> is present at the top)

```
Int a,b;
printf("enter a and b: ");  /* help message */
scanf("%d %d", &a, &b);
print("a = %d, b = %d \n",a,b);
```

The '&' symbol is needed to give scanf not the variable, but the memory location where the variable exists. Printf takes the value present at a memory. So the variable name is all that is needed. But here, we need the memory location so that scanf can write the value entered by the user into that memory.

gets and fgets:

Scanf is fine for entering numbers. But when it comes to strings, it is more difficult. Better to use gets or fgets. These are functions that read in an entire string. Usage is as follows:

```
char *gets(char *s); /* very unsafe and should not be used, as it assumes s has enough space to accept whatever string entered by the user. DO NOT USE */
char *fgets(char *s, int size, FILE *stream);
/* size is the size of s and fgets will only read upto size-1 characters and add \0
At end. Returns the string if success and NULL if error. */
```

Example: Read a name from stdin:

```
char name[10];
printf("Enter your name: ");
fgets(name,10,stdin);
```

```
printf("Name entered: %s\n",name);
```

The remainder function:

We have already looked at +, -, * and /. An additional operator is the modulus operator. x%y is defined to be the remainder when x is divided by y. Both / and % are machine dependent when the arguments are negative.

Logical operators:

Relational operators are operators that act on logical expressions to get logical answers. If i=4 and j=7, then

```
i < j  is True
2*i < j is false.
( i < j ) && ( 2*i > j ) is True as both statements are true
```

These will be very important once we talk about control statements.

Control Statements: if, for and while and switch:

```
if( logical expression ){
        Code to be executed if TRUE
}

if( logical expression ){
        Code to be executed if TRUE
}else{
        Code to be executed if FALSE
}

if( logical expression 1 ){
        Code to be executed if logical expression 1 is TRUE
}else if( logical expression 2 ){
        Code to be executed if logical expression 2 is TRUE
}else{
        Code to be executed if logical expression 1 is FALSE
}
```

IF statements can be nested. Eg from previous assignment:

```
if( D >= 0.0 ){
        if( D == 0.0 ){/* nested if */
                printf("The roots are identical \n");
                … code to find the roots' values
```

```
        } else {        /* this else goes with nested if */
                printf("The roots are real and distinct\n");
                … code to find the roots' values
        }
} else {                /* this goes with outer if statement */
        printf("The roots are complex and distinct\n");
        … code to find the roots
        }
}
```

While and for:

```
while( logical expression ){ /* starts loop */
        Code                    /* must modify logical expression to terminate the loop */
}

for( expr 1 ; logical expr 2 ; expr 3 ){
        Code                    /* executed till expr2 becomes false */
}
```

For is the same as the following code:

```
Expr 1;
while( logical expr 2 ){
        Code;
        Expr 3;
}
```

Example: print all the digits in order:

```
printf("digits: \n");
for( int i=0 ; i<10 ; i++ ){
        printf("%1d\n",i);
}
```

New additions to this section:

More Control structures:

Conditional Expression: (p 51-52 of K&R)

(expr1)? Expr 2 : expr 3;

Eg. max(a,b) is implemented in C as

(a>b) ? a : b;

Eg. printing 10 lines of an array per page:

```
for( i=0 ; i<n ; i++ )
    printf("%6d%c", a[i], (i%10==9 || i==n-1 ) ? \n : " ");
```

The %c prints blanks except when i%10 is 9 or i is n-1. In those cases inserts a \n (newpage)

Do While:

```
do{
    Code
}while( logical expression )
```

It is like while(expr){

```
    Code
}
```

except that the Code is executed at least once.

Eg. To extract the digits of a positive number s: p64 of K&R

```
    Do{
        S[i++] = n%10 + '0'     /* get next remainder and convert into ascii */
    }while( (n /= 10) > 0 );     /* divide n by 10, and loop if >0 */
```

If s is less than 10, the division yields zero and terminates too soon. So the do … while.

Break and Continue

If we want to exit the loop you are in prior to the exit condition, use BREAK. To merely skip the rest of the body of the loop and go to the next iteration, use CONTINUE.

Eg: remove trailing blanks, tabs and newlines. Line is in ‘s’.

```
for( n = strlen(s)-1 ; n>=0 ; n– ){          /* count down n from length of string */
    if( s[n] != ‘ ‘ && s[n] != ‘\t’ && s[n] != ‘\n’ )
        break;                  /* exit the for loop if we have only blanks, tabs or \n */
}                               /* indicates the loop that is skipped.*/
```

Eg: skip processing certain elements in a loop (p. 65 of K&R)

```
for( i=0 ; i<n ; i++ ){         /* loop to process elements of an array */
    if( a[i]<0 )
        continue;               /* do not process this element as it is -ve */
    ….                          /* process remaining elements */
}
```

EVEN MORE Control Commands

Bitwise Operators:

& bitwise AND	x=x & ~077	(zeros last 6 bits of x)
! bitwise (inclusive) OR	x=x ! mask	(sets ON the 1 bits of the mask)
^ bitwise (exclusive) OR	x ^ y	(sets to 1 all positions where x and y differ and 0 where they agree)

<< left shift	x << 2 shifts bits in x to left, padding with zeros	
>> right shift	x >> 2 shifts bits in x to right, padding with 0s.	
~ one's complement of the argument	~x	converts all 1 bits to 0 and 0 bits to 1

These operators have higher precedence than &&, ||, ?:, = etc but lower precedence than ==, !=, <, <=, etc

These bits operations are useful to take apart the bits of a word. For instance if we know that the 4th bit of a variable x is 1 if a certain action is to be taken (very common in operating system situations), we "test" the variable by

```
if( x & 8 ){

        …

}
```

Where 8 is 1000 in binary. x&8 returns 1 if the 4th bit of x is 1, and zero otherwise. You will use this in the first question.

FUNCTIONS

A function is defined as

```
Type FunctionName(type arg1, type arg2, …){

        Definition of the function

        …

        …

}
```

The function must be declared prior to invoking it

A function can have local variables, and can reference any external (or global) variables (variables that have been defined outside any function, including main). Note that only external variables defined in the same file as the function can be accessed. If the main function is in another file, the external variables declared in that file are NOT accessible. (see p73 of the textbook)

Functions can call other functions, and can call themselves.

The arguments of a function are “passed by value”, which means that a copy of the variable is sent to the function. This means that the argument can be modified inside the function and thrown away. The results of the function can either be returned via the return (expression) command, or by storing the answer in an object whose address has been passed to the function.

In the problems here, both the strings and arrays are passed by address. So data can be returned through them. Whereas in the third problem, the status of the answer is returned by the return command.

Pointers

Int a,b; are variables that are stored in some memory locations
Int *p; is a pointer. Right now it has no value which means it might point to any address.
If I now type:

```
a = 4;
b = 6;
p = &a;              /* This means that pointer p now holds the address */
                     /* of variable a */
b += *p      /* This is the same as b += a, since p points to variable a */
```

So, in an expression, *p refers to the value in the memory address pointed to by p.
When defining the pointer,

```
Int *p;      /* means p is a pointer of type ‘int’. */
```

Type of a pointer
The type of a pointer means the type of variable that is there at that address. A pointer could be pointing to a string:

```
char *p;
```

Or could be a pointer to double:

```
double *p;
```

Or it could be a pointer to an array of char (ie a string):

```
char *p=”This is a string”;
```

Or it could be a pointer to an array of a numeric type (say float):

```
int main(){
        float a[3],*q;
        a[0]=1.5;a[1]=2.6;a[2]=4.6;
        q=a;
        printf("q=%f, q+1=%f, q+2=%f\n",*q,*(q+1),*(q+2));
}
```

Output: q=1.500000, q+1=2.600000, q+2=4.600000

Pointer arithmetic:

If p points to a, then p+1 points to the next memory location to ‘a’, of the “type” declared for ‘a’. So since ‘a’ has been declared float, q+1 will point to an address that is one float beyond the address of a[0], i.e. it points to a[1]. Etc. So p+i points to a[i].

This makes pointers very powerful. All of array manipulation can now be done using pointer arithmetic. **Beware:** I can say

```
int main(){
        float a[3],*q;
        a[0]=1.5;a[1]=2.6;a[2]=4.6;
        q=a;
        printf("q=%f, q+1=%f, q+2=%f, q+3=%f\n",*q,*(q+1),*(q+2),*(q+3));
}
```

Here is what happens when I compile and run:

```
$ gcc ptr1.c            /* compiles without error */
$ ./a.out
q=1.500000, q+1=2.600000, q+2=4.600000, q+3=-36141.00 /* gives value for q+3! */
$ ./a.out               /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000,
q+3=1925517263273987271557l2.000000                    /* different answer! */
$ ./a.out               /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000,
q+3=-11804037870771613760010517092440735744.000000  /* different answer! */
$ ./a.out               /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000, q+3=-0.054069 /* different answer! */
```

Summary:

```
Int *ip;        /* ip is a pointer to int */
Ip = &x;        /* ip now points to x (say it holds 1) */
y = *ip;        /* y is now 1 */
*ip = 0;        /* x is now set to 0 */
Ip = &z[0];     /* ip now points to z[0] */

y = *ip + 1;    /* *ip has highest precedence, so y = (*ip) + 1 */
*ip += 1;       /* increments z[0] by 1 */
```

However note that sometimes ( ) are needed:

```
++*ip;          /* also increments z[0] */
(*ip)++;        /* also increments z[0] */
*ip++;          /* equivalent to *(ip++) since ++ has the higher precedence */
```

Pointers (contd):

Special gotchas:
If a is an array and pa a pointer, we can do the following::

```
pa = a;
pa++;
```

/* But we cannot then do the following */

```
a=pa; /* wrong! */
a++;  /* wrong! */
```

/* these lines are wrong since a is a constant pointer. It is tied to &a[0] */

Now since a is a constant pointer, it can be passed to a function as an argument. (what is passed is a copy of the pointer.) So we can get the length of a constant array:

```
strlen("hello, world\n");      /* ok, returns length of string constant */
strlen(a);                     /* a is a character array a[100]; */
strlen(ptr);                   /* char *ptr; */
```

We can even modify the contents of a in a function, but not the contents of a string constant. Whether it is flagged or it is merely wrong I am not sure

Consider:

```
char a[100];  /* character array */
char *p;      /* pointer */
p=a+2;        /* p points to a[2] */
…
n=f(a+2);     /* a+2 is passed as an argument to function f */
```

In the function, this is declared as

```
Int f(char arr){
        Int n;
        n = arr[3];     /* refers to (arr +3) = a+5 */
        n = arr-2;      /* refers to (arr -2) = a+2 -2 = a */
}
```

So using pointer arithmetic, negative indices are allowed. But it is your responsibility to ensure that the array references are within bounds.

Last assignment we converted strlen from an index based function to a pointer based one. Let us see what changed between the three versions that K&R give for strlen (on pages 39, 99 and 103):

```
/* strlen v 1 */
Int strlen(char s[]){
   int i;
   I = 0;
   while( s[i] != '\0' )
      ++i;
   return i;
}
```

```
/* strlen v 2 */
int strlen(char *s){
   int n;

   for( n=0 ; *s != '\0' ; s++ )
      n++;
   return n;
}
```

```
/* strlen v 3 */
int strlen(char *s){
   char *p = s;

   while( *p != '\0' )
      p++;
   return p-s;
}
```

All three functions do the same thing. But v1 treats s as an array. Versions 2 and 3 treat s as a pointer. Which is correct? Both are correct.

Note the use of pointer arithmetic in v 3: length was computed as p-s. Why is this correct?

Pointers arguments can be used as local variables (p 105 of K&R):

```
Void strcpy( char *s, char *t ){
        while ((*s = *t ) != '\0' ){
                s++;                    /* pointer s is modified */
                t++;                    /* pointer t is modified */
        }
}
```

The reason is that *s and *p were passed by value. So the pointer in the calling program is not lost. (Compare the three versions of the strcpy function on the page.)

Arrays of pointers: (pages 108, 109)

```
#define MAXLINES 5000
#define MAXLEN 80
char line[MAXLEN];          /* a character array of length 80 */
char *p;                    /* pointer to char (not pointing to anything */
char *lineptr[MAXLINES];    /* array of pointers */
```

Note that this is a 5000 element array of pointers, **but none of the pointers is pointing to memory as yet.** Suppose we have a function getline(char *line, int max); that reads in a line from the terminal, given an argument that can hold the line. Now we read in the lines into our *lineptr array:

```
#include <stdio.h>
#include <string.h>
#define MAXLINES 5000
char *lineptr[MAXLINES];          /* array of pointers declared */
….
….
```

```
if (( nlines = readlines(lineptr,MAXLINES))>= 0){ /* call readlines to read the lines */
….
Int readlines(char *lineptr[], int maxlines){
        char *p, line[MAXLEN];     /* MAXLEN is the max length of a line allowed */
        ….
        while(((len = getline(line,MAXLEN))>0)   /* read in one line at a time */
        ….
                p = alloc(len)                          /* allocate a buffer of length len */
                                                        /* and assign to p */
                strcpy(p, line);                        /* copy line into p buffer*/
                lineptr[nlines++] = p;                  /* assign lineptr[nlines] to p buffer */
        …
…
```

Each time a line is copied from input, it is stored in line. But line will be overwritten next time getline is called again. So we **allocate** a buffer (**no name yet, it is just some memory)** and have p point to it, and copy line into buffer p. Finally, lineptr[nlines] is assigned to p, so that p can be freed and be reused.

So we have two types of arrays: arrays whose memory requirement is allocated at the beginning:

```
int a[10];
```

and arrays whose memory is allocated while the program is running:

```
char *lineptr[5000];
```

whose memory has to be allocated as required.

So what is the command used to allocate the memory? Look up “man malloc”

```
void malloc(size_t size)      /* size in bytes */
```

Will return a size that was asked for. The size can be in bytes or can be num*sizeof(float) etc.

2-D Arrays

C has different types of 2D arrays of which two types are widely used. We will only look at those two.

Static arrays:

```
float a[10][15];          /* Array a has 10 rows and 15 columns. */
A[3][5] = 3.5;            /* assignment into the array */
f(float a[6][15]){...}
```

```
f(float a[][15]){...}        /* function call only needs to specify the number of columns) */
Or
f(float (*a)[15]){...}       /* we treat a[6] to be a pointer attached to an array. */
```

All are valid.

Pointer arrays:

```
Float *pa[15];          /* only the pointers to each row are declared. The rest needs to be */
                        /* allocated. This is the same as the previous case */
```

So main difference is that when passing a static array as float (*a)[15], we are passing a pointer to the specific row of the array. The row already exists.

When passing the same via a pointer array, we use *pa[15], but the row may not even exist.

Function Pointers:

A function can take arguments, which can be pointers. But an added feature is that a function can have as argument, the pointer to another function. For example in Problem 3 of Assignment 8, function trapzd takes as its first argument a pointer to a function that calculates 1.0/(1.0+x*x). How is this pointer calculated? The function to be passed to trapzd is declared as

```
float fn(float x)
```

A pointer to this function is then

```
float (*fn)(float x)
```

This is then passed to the trapzd function as:

```
float trapzd( float (*func) (float x), float a, float b, int n )
```

Note that trapzd does not know WHICH function you are giving it to integrate. So it gives it a generic name, func. The main program knows the function, and it passes it to trapzd:

```
float myfun(float x){
        return 1.0/(1.0+x*x);
}
…
intg=trapzd(myfun,a,b,n)
```

I am adding below something from Stackoverflow, which makes things very clear.:

A prototype for a function which takes a function parameter looks like the following:

```
None
void func ( void (*f)(int) );
```

This states that the parameter `f` will be a pointer to a function which has a `void` return type and which takes a single `int` parameter. The following function (`print`) is an example of a function which could be passed to `func` as a parameter because it is the proper type:

```
None
void print ( int x ) {
 printf("%d\n", x);
}
```

Function Call

When calling a function with a function parameter, the value passed must be a pointer to a function. Use the function's name (without parentheses) for this:

```
None
func(print);
```

would call `func`, passing the print function to it.

Function Body

As with any parameter, `func` can now use the parameter's name in the function body to access the value of the parameter. Let's say that `func` will apply the function it is passed to the numbers 0-4. Consider, first, what the loop would look like to call print directly:

```
None
for ( int ctr = 0 ; ctr < 5 ; ctr++ ) {
 print(ctr);
}
```

Since `func`'s parameter declaration says that `f` is the name for a pointer to the desired function, we recall first that if `f` is a pointer then `*f` is the thing that `f` points to (i.e. the function `print` in this case). As a result, just replace every occurrence of print in the loop above with `*f`:

None

```
void func ( void (*f)(int) ) {
 for ( int ctr = 0 ; ctr < 5 ; ctr++ ) {
  (*f)(ctr);
 }
}
```

Structures, arrays and pointers:

Structures are free form collections of elements. While an array can hold a set of elements that are all of one type, structures can hold anything (except functions), including pointers, arrays and other structures.

Egs: A pixel in the screen:

```
struct point{
        int x;   /* x location of pixel */
        int y;   /* y location of pixel */
};      /* can directly define variables of this type */

struct point pt1, pt2;          /* declares two variables of type struct point */

struct point maxpt = {320,200};     /* can also initialize the variable */
```

To refer to a member of a structure variable, we use

```
printf("%d,%d", pt1.x, pt1.y);        /* prints out the coordinates of point pt1 */

double dist, sqrt(double);            /* sqrt can be taken from math.h as well */
dist = sqrt((double)pt.x*pt.x + (double)pt.y*pt.y);
```

here we use the fact that C will promote the second pt.x to double automatically

We now use a struct to build another struct:

```
Struct rect{
        struct point pt1;
        struct point pt2;
};

Now we can define
struct rect screen;             /* used to define the size of the screen in pixels */
```

Structures and Functions:

```
struct point makepoint(int x, int y){
        struct point temp;    /* declare a local struct point variable */

        temp.x =x;
        temp.y =y;
        return temp;          /* returns the structure to calling program */
}
```

Use it to initialize other structures (this is a part of the calling program)

```
struct rect screen;
struct point middle;
struct point makepoint(int, int);

screen.pt1 = makepoint(0,0);                  /* return val of makepoint(0,0) used to initialize
                                                 screen.pt1 */
screen.pt2 = makepoint(XMAX,YMAX);    /* pt2 is top right corner of screen */

middle = makepoint( (screen.pt1.x + screen.pt2.x)/2,
                          (screen.pt1.y + screen.pt2.y)/2);
```

Function to find if point p is inside a rect r:

```
int ptinrect(struct point p, struct rect r){
        return p.x >= r.pt1.x && p.x < r.pt2.x
            && p.y >= r.pt1.y && p.y < r.pt2.y;
}
```

Pointer to structure:

```
struct point origin, *pp;       /* origin is a structure variable, pp is a pointer to struct point*/
pp = &origin;                   /* pp now points to origin */
printf("origin is (%d,%d)\n", (*pp).x,(*pp).y );       /* (..) needed to get origin.x and origin.y*/
```

pp points to origin. (*pp).x gives x value. Can instead write pp->x. I.e.,

```
printf("origin is (%d,%d)\n", pp->x, pp-y );          /* also points to origin.x and origin.y */
```

NOTE: Highest precedence is for ., ->, () and []. So, if we have

```
struct node{
        int len;
        char *str;
} *p;
```

Then ++p->len means ++(p->len) and not (++p)->len. So be very careful in how the different operators combine when using . and ->

Arrays of structures

```
struct key{
        char *word;
        int count;
} keytab[NKEYS];            /* keytab is an array of structures*/
```

Structures containing pointers:

We want a structure that defines a "node" that contains

A pointer to the text of a word
A count to the number of occurrences that word
A pointer to the left child node
A pointer to the right child node

This is very common in algorithms that use binary trees (you will learn about this when you study data structures). It is defined as:

```
struct node {
        char *word;
        int count;
        struct node *left;      /* pointer to left child */
        struct node *right;     /* pointer to right child */
};
```

So struct node is defined in terms of struct node *left and struct node *right. But these are only pointers to node, and it is ok to have what looks like a reference to the node itself.

Alternately we want to have a chain of people who are in a project. We could define a "link":

```
struct member{
        int id;             /* id of the person */
        char *name;   /* name of the person (just as an example) */
        struct member *prev;          /* link to previous member */
        struct member *next; /* link to next member */
}
```

The list starts with

Struct member top = { 0 , NULL, NULL, NULL };

This means that there is no member in the chain. We add a member with id 15, and name containing "Esther". To do that we need to allocate memory and initialize it:

```
top.next = (struct member *)malloc(sizeof(struct member));
                                                    /* top.next points to the memory */
```

```
top.next.id = 15;
top.next.name = (char *)malloc(sizeof name);
strcpy(top.next.name,name);        /* copy the name into the name field of the link */
top.next.prev = top;               /* this new link must point to top */
top.next.next = NULL;              /* link has no next link yet */
…
```

This continues, till at some point, we need to delete a node, say, p. We have to connect next to prev before deleting p itself:

```
(p.prev).next = (p.next);
(p.next).prev = (p.prev);
```

Now we can delete the link itself. But before we can delete the link, we must free the memory used to store the name.

```
free(p.name);
free(p);
```

If we do not free p.name, that piece of memory remains allocated, but with nothing pointing to it. So it will remain allocated till the computer is rebooted. Such errors cause the memory to get filled with such fragments of memory till the computer is no longer able to allocate any new memory and the computer crashes.

EE1102 Introduction to Programming **Assignment 9**

Total Marks: 100

Due Date and Time:

Submission Procedure: Upload the C program files by the due date and time. The files should be named as specified in each problem statement. Replace ROLLNO with your roll no (all small letters). **Do not upload exe files.**

All programs should be readable with adequate comments and documentation.

This assignment is about a data base describing the employees of an IT company with 1000 employees. We will define the (very simplified) structure of the company and then first attempt to

A) use arrays, then
B) arrays of structures with pointers, then
C) arrays of structures with linked lists (with pointers …)

The goal is to be able to answer the following questions:

Q1: How many programmers work in Proj 1? How many man-hours are allocated to Proj 2?
Q2: Who reports to Manager X?
Q3: Who is W's boss?
Q4: How many programmers are there in Proj 1?
Q5: Which secretaries do not report to anyone (ie., are jobless)?
Q6: Which employees have no one reporting to them?

(W and X are simply dummy names)

Any database should be able to add members, and also delete members. Those capabilities should be added in each of A - E. At the end of this very simple database, you should be able to see the strengths and weaknesses of these different

Problem 1 (Arrays): Write a C program called ROLLNO_arrays.c that opens a file containing the information about all employees and inputs this information into one or more arrays. Each employee is described by the following fields:

id, name, status, job title, rating, list of people + projects they are working in, and corresponding hours per week.

Note: status means if the person is active or has resigned. (employees are never removed from the list. They are only active or inactive)

Example:

id	status	name	Job title	rating	projects/people	hours
135	active	anjali	manager	3	1	10
135	active	anjali	manager	4	2	10
146	active	arun	programmer	3	1/135	40
146	inactive	arun				
184	active	hariharan	secretary	4	1/110	10
184	active	hariharan	secretary	4	2/110	15
110	active	ganesh	programmer	4	1/135	10
110	active	ganesh	programmer	4	2/135	20
148	active	tarun	secretary	2	1/146	40
110	active	ganesh	programmer	4	3/135	10

So, arun is in project 1, but for some reason is inactive (perhaps on leave, or has resigned) Use arrays to hold this data. Each id should correspond to a single row. How will you handle people in multiple projects? (Can have a space separated list of ids and another such list of projects. But which id goes with which project?)

Print out the answers to Q1 to Q5

Problem 2 (arrays of structures): Write a C program called ROLLNO_array_of_structures.c reads same file as above, and builds up an array whose elements are structures.Each field of the table will now correspond to a member of the structure. In addition you can have two more members of the structure that indicate the person and project of upto two projects, but no linked lists. Assume that there are at most two projects, and at most a person reports to two people.

Print out the answers to Q1 to Q5

Problem 3 (arrays of structures with linked lists): Write a program called ROLLNO_arrays_of_structures_lists.c reads same file as above, and builds up an array whose elements are structures. But now, more than two projects can be linked via a linked list, each of whose links are structures, giving the person and project.

Print out the answers to Q1 to Q5